

ASSIGNMENT NO.3

Question 1 — Network Request Dispatcher

You are building a backend HTTP client library for an internal microservices platform. The team uses magic integers like 200, 404, 500 all over the codebase. A senior engineer raises a code review comment:

"These raw integers are impossible to maintain. If someone passes 500 where 404 was expected, the compiler won't catch it. We need strongly-typed status codes — and we need proper type aliases because writing `std::vector<std::pair<std::string, std::string>>` for HTTP headers everywhere is unreadable."

Part A — enum class for HTTP Status Codes

Define the following enum class:

Question 1 — Network Request Dispatcher

OK	= 200,
Created	= 201,
BadRequest	= 400,
Unauthorized	= 401,
NotFound	= 404,
ServerError	= 500

Write a function:

```
void handleResponse(HttpStatus status, const string& endpoint);
```

Use a **switch** statement inside **handleResponse()** to print a message for each status code. Since **enum class** values cannot be used directly in **switch** without explicit handling, use **static_cast(status)** to compare.

In **main()**, call **handleResponse()** with at least 4 different status codes.

Code:

```
#include <iostream>
#include <string>
using namespace std;

enum class HttpStatus
{
    OK = 200,
    Created = 201,
    BadRequest = 400,
    Unauthorized = 401,
    NotFound = 404,
    ServerError = 500
};

void handleResponse(HttpStatus status, const string& endpoint)
```

```

{
    switch (static_cast<int>(status))
    {
        case 200:
            cout << "[GET " << endpoint << "]" -> 200 OK : Request successful" << endl;
            break;

        case 201:
            cout << "[POST " << endpoint << "]" -> 201 Created : Resource created" << endl;
            break;

        case 400:
            cout << "[GET " << endpoint << "]" -> 400 Bad Request : Invalid request" << endl;
            break;

        case 401:
            cout << "[POST " << endpoint << "]" -> 401 Unauthorized : Authentication required" << endl;
            break;

        case 404:
            cout << "[GET " << endpoint << "]" -> 404 Not Found : Endpoint does not exist" << endl;
            break;

        case 500:
            cout << "[POST " << endpoint << "]" -> 500 Server Error : Internal server error - retry later" << endl;
            break;
    }
}

int main()
{
    handleResponse(HttpStatus::OK, "/api/users");
    handleResponse(HttpStatus::Unauthorized, "/api/login");
    handleResponse(HttpStatus::NotFound, "/api/products/99");
    handleResponse(HttpStatus::ServerError, "/api/order");

    return 0;
}

```

Expected Output:

```

[GET /api/users] -> 200 OK : Request successful
[POST /api/login] -> 401 Unauthorized : Authentication required
[GET /api/products/99] -> 404 Not Found : Endpoint does not exist
[POST /api/order] -> 500 Server Error : Internal server error - retry later

```

Part B — Type Aliases for Readability

The team's request/response headers are stored as a list of key-value pairs. Define the following type aliases using the using keyword:

```
using Header = pair<string, string>;
using HeaderList = vector<Header>;
using Port = unsigned int;
using IPAddress = string;
```

Also define a traditional **typedef** alias:

```
typedef unsigned long long RequestId;
```

Write a function `printHeaders(const HeaderList& headers)` that prints all key-value pairs. In `main()`, create a `HeaderList` with at least 3 headers and call `printHeaders()`. Also declare a `Port` variable for the server port and a **RequestId** to track the request.

Codes:

```
#include <iostream>
#include <string>
#include <vector>
#include <utility>
using namespace std;
using Header = pair<string, string>;
using HeaderList = vector<Header>;
using Port = unsigned int;
using IPAddress = string;

typedef unsigned long long RequestId;

void printHeaders(const HeaderList& headers)
{
    for (const Header& header : headers)
    {
        cout << header.first << " : " << header.second << endl;
    }
}

int main()
```

```
{
    HeaderList headers = {
        {"Content-Type", "application/json"},
        {"Authorization", "Bearer eyJhbGci..."},
        {"Accept-Language", "en-US"}
    };
    Port serverPort = 8080;

    RequestId requestId = 1748293847;

    cout << "Request ID : " << requestId << endl;
    cout << "Server Port : " << serverPort << endl;

    cout << endl;
    cout << "Headers:" << endl;

    printHeaders(headers);
    return 0;
}
```

Expected Output:

Request ID : 1748293847

Server Port : 8080

Headers:

Content-Type : application/json

Authorization : Bearer eyJhbGci...

Accept-Language : en-US

Question 2 — Game Asset Manager

You have joined the tools team at a game studio. The asset pipeline loads textures, shaders, and audio clips. A previous intern wrote the asset loader using raw new and delete — and the game crashes every few runs with a double-deletion error in the shader module.

Your tech lead says:

"Raw pointers here are the root problem. A texture can be owned by only one system at a time — that's `unique_ptr`. Shaders are shared between the renderer and the editor — that's `shared_ptr`. Audio clips are observed by the event system but the event system must not keep them alive if they're unloaded — that's `weak_ptr`. Rewrite it."

Part A — `unique_ptr` : Exclusive Asset Ownership

Write a class `Texture` with:

- Private: `string name`, `int width`, `int height`
- Constructor prints: `[Texture Loaded]`
- Destructor prints: `[Texture Released]`
- `void display() const` — prints dimensions

In `main()`:

1. Create a `Texture` using `make_unique("player_sprite", 512, 512)`
2. Call `display()` through the `unique_ptr`
3. Attempt to copy the `unique_ptr` — comment it out and write a one-line explanation of the error
4. Transfer ownership to a second `unique_ptr` using `std::move()` — verify the first is now `nullptr`
5. Let it go out of scope — observe destructor

// Required structure in `main()`:

```
auto tex1 = make_unique<Texture>("player_sprite", 512, 512);
tex1->display();
```

```
// unique_ptr<Texture> tex2 = tex1; // ← Comment this out, explain why it fails
```

```
unique_ptr<Texture> tex2 = move(tex1);
cout << "tex1 is null: " << (tex1 == nullptr ? "YES" : "NO") << endl;
```

Code:

```
#include <iostream>
#include <string>
#include <memory>

using namespace std;
class Texture
{
private:
    string name;
    int width;
    int height;

public:
    Texture(string n, int w, int h)
    {
        name = n;
        width = w;
        height = h;

        cout << "[Texture Loaded]" << endl;
    }

    ~Texture()
    {
        cout << "[Texture Released]" << endl;
    }

    void display() const
    {
        cout << "Texture: " << name << endl;
        cout << "Dimensions: " << width << " x " << height << endl;
    }
};

int main()
{
    auto tex1 = make_unique<Texture>("player_sprite", 512, 512);
    tex1->display();
    unique_ptr<Texture> tex2 = move(tex1);
    cout << "tex1 is null: "
        << (tex1 == nullptr ? "YES" : "NO") << endl;
    tex2->display();
    return 0;
}
```

```
}
```

Output:

```
[Texture Loaded]
Texture: player_sprite
Dimensions: 512 x 512
tex1 is null: YES
Texture: player_sprite
Dimensions: 512 x 512
[Texture Released]
```

Part B — shared_ptr : Shared Shader Ownership

Write a class Shader with:

- Private: string name, string type ("vertex" / "fragment")
- Constructor prints: [Shader Compiled]
- Destructor prints: [Shader Destroyed]
- int getReferenceCount() — returns use_count() via a shared_ptr passed as parameter (see below)

In main():

1. Create a Shader using make_shared("main_vert", "vertex")
2. Assign the same shared_ptr to two more variables (simulating renderer + editor sharing it)
3. Print the reference count after each assignment — must show 1, then 2, then 3
4. Let one go out of scope inside a block {} and print the count again — it should drop

// Required structure in main():

```
auto shader = make_shared<Shader>("main_vert", "vertex"); cout << "Ref count: " << shader.use_count()
<< endl;          // 1
{
    auto rendererRef = shader;
    cout << "Ref count: " << shader.use_count() << endl; // 2
    auto editorRef = shader;
    cout << "Ref count: " << shader.use_count() << endl; // 3
}
cout << "Ref count: " << shader.use_count() << endl;    // 1
```

Code:

```
#include <iostream>
#include <string>
#include <memory>

using namespace std;
class Shader
{
private:
    string name;
    string type;

public:
    Shader(string n, string t)
    {
        name = n;
        type = t;

        cout << "[Shader Compiled]" << endl;
    }

    ~Shader()
    {
        cout << "[Shader Destroyed]" << endl;
    }

    int getReferenceCount(const shared_ptr<Shader>& shader)
    {
        return shader.use_count();
    }
};

int main()
{
    auto shader = make_shared<Shader>("main_vert", "vertex");
    cout << "Ref count: " << shader.use_count() << endl;
    {
        auto rendererRef = shader;
        cout << "Ref count: " << shader.use_count() << endl;
        auto editorRef = shader;
        cout << "Ref count: " << shader.use_count() << endl;
    }

    cout << "Ref count: " << shader.use_count() << endl;
```



```
    return 0;
}
Output:
[Shader Compiled]
Ref count: 1
Ref count: 2
Ref count: 3
Ref count: 1
[Shader Destroyed]
```

Part C — weak_ptr : Non-Owning Observer

The audio event system needs to observe whether an AudioClip is still loaded without preventing it from being unloaded. If the clip has been released, the event system must handle it gracefully.

In main():

1. Create an AudioClip using make_shared("explosion", 3.5)
2. Assign a weak_ptr from the shared_ptr
3. Use lock() on the weak_ptr to access the clip — print its name if still alive
4. Reset the shared_ptr (simulating unload) — audio.reset();
5. Try lock() again — use expired() to check — print "Clip already unloaded" if gone

// Required structure in main():

```
auto audio = make_shared<AudioClip>("explosion", 3.5);
weak_ptr<AudioClip> observer = audio;
```

```
if (auto clip = observer.lock())
    cout << "Clip alive: " << clip->getName() << endl;
audio.reset();    // unload
```

```
if (observer.expired())
    cout << "Clip already unloaded." << endl;
```

Code:

```
#include <iostream>
#include <string>
#include <memory>

using namespace std;
class AudioClip
{
private:
    string name;
    double duration;

public:
    AudioClip(string n, double d)
    {
        name = n;
        duration = d;

        cout << "[AudioClip Loaded]" << endl;
    }

    ~AudioClip()
    {
        cout << "[AudioClip Destroyed]" << endl;
    }
    string getName() const
    {
        return name;
    }
};

int main()
{
    auto audio = make_shared<AudioClip>("explosion", 3.5);
    weak_ptr<AudioClip> observer = audio;
    if (auto clip = observer.lock())
    {
        cout << "Clip alive: " << clip->getName() << endl;
    }
    audio.reset();

    if (observer.expired())
    {
        cout << "Clip already unloaded." << endl;
    }
}
```

```

    }

    return 0;
}
Output:
[AudioClip Loaded]
Clip alive: explosion
[AudioClip Destroyed]
Clip already unloaded.

```

Question 3 — Log Buffer with Proper Copy Semantics

Your team is building a high-performance logging library. A LogBuffer class dynamically allocates a char array to store log messages. A junior developer copies a LogBuffer object and then both copies go out of scope — the program crashes with a double-deletion error.

"This is a classic shallow copy problem. The compiler's default copy constructor just copies the pointer — both objects end up pointing to the same memory. When the first one is destroyed, the memory is freed. When the second one is destroyed, it frees already-freed memory. You need to implement the Rule of Three."

Design class LogBuffer:

Private data members:

- char buffer* — dynamically allocated character array
- int capacity — maximum number of characters
- int size — number of characters currently stored
- static int instanceCount — tracks total live LogBuffer objects

Public interface:

// Constructor — allocates char buffer of given capacity on heap**

LogBuffer(int capacity);

// Copy Constructor — must perform a DEEP copy LogBuffer(const LogBuffer& other);

// Copy Assignment Operator — must handle self-assignment, release old memory, deep copy
LogBuffer& operator=(const LogBuffer& other);

// Destructor — frees the buffer, decrements instanceCount

~LogBuffer();

void append(const char* msg);// Appends msg to buffer (up to capacity)

```

void print() const; // Prints current buffer contents
void clear(); // Resets size to 0

static int getInstanceCount(); // Returns current live object count

main() — Required structure:
int main() {**

// Objective 1 : Basic usage
LogBuffer log1(256);
log1.append("Server started on port 8080");
log1.append(" | Request received from 192.168.1.10");
log1.print();

// Objective 2 : Deep copy via copy constructor
LogBuffer log2 = log1; // copy constructor
log2.append(" | Cached response sent");
cout << "log1 : "; log1.print(); // must NOT contain log2's append
cout << "log2 : "; log2.print();

// Objective 3 : Copy assignment operator
LogBuffer log3(128);
log3 = log1; // copy assignment
log3.print();

// Objective 4 : Self-assignment guard
log1 = log1; // must not crash or corrupt data
log1.print();
// Objective 5 : Static member
cout << "Live LogBuffer objects : " << LogBuffer::getInstanceCount() << endl;
return 0;
// All 3 objects destroyed here — destructors must print and decrement count
}

```

Requirements:

- The copy constructor must allocate a new char array of the same capacity and copy the contents using memcpy or a character loop — not just buffer = other.buffer
- The copy assignment operator must check for self-assignment (if (this == &other)), delete[] the existing buffer before allocating the new one
- The destructor must delete[] the buffer and print a message
- instanceCount must be incremented in every constructor and decremented in the destructor

Code:

```
#include <iostream>
#include <cstring>
using namespace std;
class LogBuffer
{
private:
    char* buffer;
    int capacity;
    int size;
    static int instanceCount;
public:

    LogBuffer(int capacity)
    {
        this->capacity = capacity;
        size = 0;
        buffer = new char[capacity];
        buffer[0] = '\0';
        instanceCount++;
        cout << "[LogBuffer Created] capacity=" << capacity << endl;
    }

    LogBuffer(const LogBuffer& other)
    {
        capacity = other.capacity;
        size = other.size;
        buffer = new char[capacity];
        memcpy(buffer, other.buffer, size + 1);
        instanceCount++;
        cout << "[LogBuffer Deep Copied] capacity=" << capacity << endl;
    }

    LogBuffer& operator=(const LogBuffer& other)
    {
        if (this == &other)
        {
            cout << "[Self-assignment detected - no operation]" << endl;
            return *this;
        }

        delete[] buffer;
```

```

        capacity = other.capacity;
        size = other.size;
        buffer = new char[capacity];
        memcpy(buffer, other.buffer, size + 1);

        cout << "[LogBuffer Assigned]" << endl;

        return *this;
    }

~LogBuffer()
{
    delete[] buffer;

    instanceCount--;

    cout << "[LogBuffer Destroyed]" << endl;
}

void append(const char* msg)
{
    int msgLength = strlen(msg);
    int available = capacity - size - 1;

    if (msgLength > available)
    {
        msgLength = available;
    }

    memcpy(buffer + size, msg, msgLength);
    size = size + msgLength;

    buffer[size] = '\0';
}

void print() const
{
    cout << buffer << endl;
}

void clear()
{
    size = 0;
    buffer[0] = '\0';
}

```

```

    }

    static int getInstanceCount()
    {
        return instanceCount;
    }
};

int LogBuffer::instanceCount = 0;

int main()
{
    LogBuffer log1(256);

    log1.append("Server started on port 8080");
    log1.append(" | Request received from 192.168.1.10");

    log1.print();

    LogBuffer log2 = log1;

    log2.append(" | Cached response sent");

    cout << "log1: ";
    log1.print();

    cout << "log2: ";
    log2.print();

    LogBuffer log3(128);

    log3 = log1;

    cout << "log3: ";
    log3.print();

    log1 = log1;
    log1.print();

    cout << "Live LogBuffer objects : "
        << LogBuffer::getInstanceCount() << endl;
    return 0;
}

```

Expected Output:

[LogBuffer Created] capacity=256

Server started on port 8080 | Request received from 192.168.1.10

[LogBuffer Deep Copied] capacity=256

log1: Server started on port 8080 | Request received from 192.168.1.10

log2: Server started on port 8080 | Request received from 192.168.1.10 | Cached response sent

[LogBuffer Created] capacity=128

[LogBuffer Assigned]

log3: Server started on port 8080 | Request received from 192.168.1.10

[Self-assignment detected - no operation]

Server started on port 8080 | Request received from 192.168.1.10

Live LogBuffer objects : 3

[LogBuffer Destroyed]

[LogBuffer Destroyed]

[LogBuffer Destroyed]